B.TECH RESEARCH METHODOLOGY & PROJECT BLUEPRINT WORKSHEET

Student Name / Roll No: Shlok (UAI05BTA2317), Jiya (UAI05BTA2313), Caleb (UAI05BTA2319)

Branch: BTech AI ML

Title: Neighbourhood-aware misbehavior detection for V2X networks (VeReMi Extension benchmark)

PART 1: PROBLEM CONCEPTION & MOTIVATION

1. **Relevance (What practical, industrial, or social pain point does this address?):**
   Connected cars broadcast their position, speed and heading several times a second, and nearby cars act on it (collision warnings, platooning). If a vehicle sends false data, for example a spoofed position or a frozen speed, others can be misled, and an approaching car can be hidden. Cryptographic signatures prove who sent a message, not that its content is true, so vehicle makers and road authorities need on-board misbehavior detection as a second line of defence. Social impact: road safety.
2. **Interest (Why are you personally invested or curious about solving this specific problem?):**
   Road safety is a concrete place where a machine-learning error has real consequences, which makes it a good test of whether a model can be trusted. We were curious whether a car can catch a lying vehicle the way a person would, by noticing that it disagrees with what everyone around it reports, and whether that still works when some of those neighbours are lying too.

PART 2: DATA STRATEGY

3. **Data Availability (Where will you source your data? Provide repository or collection method):**
   Public benchmark **VeReMi Extension**, downloaded from Zenodo (record 20090854, CC-BY 4.0). It is generated by simulation (F2MD / VEINS / SUMO, Luxembourg traffic scenario, 30% attackers, 19 attack types), so no data collection is needed. Used so far: RandomPos and ConstSpeed (14:00-16:00). Planned: more attacks, dense 07:00-09:00 subsets, MixAll_0024, and VeReMi NextGen (Zenodo 19665762) as an external check. Code, logs and weights: our GitHub repository.
4. **Choice of Data (What specific features/attributes will you include, and what will you filter out?):**
   Included (8 per sender per second): position relative to the receiving car (scaled by the 300 m range), speed, heading (cos, sin), position change since the last second (dx, dy), and a message-present flag. Each sample is a receiver's view: the 16 nearest senders over 10 seconds. Label: attacker or honest, taken from the attack id in the trace file name.

   Filtered out: sender identity (so the model cannot memorise vehicles), the simulated sensor-noise fields, windows where the receiver's own state is missing, and senders beyond the 16 nearest.

PART 3: METHODOLOGY & ANALYSIS

5. **Analysis of Data (Outline your ML pipeline: preprocessing, missing value handling, model selection):**
   - **Preprocessing:** parse the JSON logs, bin messages to 1-second steps, build windows every 3 seconds, compute the 8 features.
   - **Missing values:** a sender not heard in a step is zero-filled and marked by the presence flag; the model masks those steps.
   - **Model selection:** baseline = per-sender transformer; ablation = sees neighbours without cross-sender attention; proposed = factorised temporal then spatial transformer. Trained with weighted binary cross-entropy (AdamW, one-cycle learning rate, 5 epochs, batch 128); alarm threshold chosen on validation data to maximise F1; 3 seeds.

PART 4: VALIDATION, GENERALIZATION, & INTERPRETATION

6. **Generalization (How will you test if your model works beyond your specific training dataset?):**
   - **Done:** train on the first hour, test on the later hour (unseen time), with receiver-disjoint validation and test blocks.
   - **Planned:** leave-one-attack-out; train on low-density, test on rush-hour traffic; the full-day MixAll_0024 set; the separate VeReMi NextGen dataset; a colluding-attacker stress test.
7. **Analysis Validity Testing:**
   Null Hypothesis (H0): the neighbour-aware detector has the same F1 as the per-sender baseline (F1 full = F1 per-sender). Alternative (H1): it is higher.

   Evaluation Metrics: precision, recall, F1, false-positive rate, AUC-ROC, and detection latency (seconds to first alarm); also recall by share of malicious neighbours.

   Statistical Test: planned at $p < 0.05$: paired bootstrap confidence interval on the F1 difference, Wilcoxon signed-rank test across folds/seeds, McNemar's test on paired decisions, Holm-Bonferroni correction across attacks. So far we have only mean and standard deviation over 3 seeds (no p-value yet).
8. **Interpretation (What actionable engineering or real-world insight will your final results yield?):**
   - Preliminary (2 attacks, 3 seeds): neighbour context raises F1 from 0.973 to 0.982 and recall from 0.959 to 0.975 at the same false-alarm rate (about 0.5%), and cuts mean time to first alarm from 0.70 s to 0.15 s.
   - Recall stays near 0.97 even when over half of a sender's neighbours are malicious (0.971 vs 0.973 clean).
   - Insight: use neighbour context as a second line of defence behind message authentication, but check each attack (ConstSpeed was slightly worse: 0.948 vs 0.960). Limits: simulated data, independent attackers, no collusion yet.